Neo Solidity Compiler Design Specification

R3E Network

November 20, 2025

Contents

1 Scope and Purpose **1**
1.1 Objectives . . . 1

2 References and Terminology **2**
2.1 Normative References . . . 2
2.2 Terminology . . . 2

3 System Objectives and Constraints **3**

4 Architectural Overview **3**
4.1 Stage Contracts . . . 4

5 Component Specifications **4**
5.1 Frontend Integration . . . 4
5.2 Canonical IR and Normalizer . . . 4
5.3 Semantic Analysis . . . 5
5.4 Optimization Engine . . . 5
5.5 NeoVM Code Generator . . . 5
5.6 Runtime Manager and Artifact Builder . . . 5
5.7 Tooling Surfaces . . . 6
5.8 Hardhat Integration Example . . . 6
5.9 Foundry Integration Notes . . . 6
5.10 DevPack Interface Summary . . . 7
5.11 DevPack API Details . . . 7

6 End-to-End Compilation Detail **8**
6.1 Solidity Source to Canonical IR . . . 8
6.2 Yul-to-NeoVM Conversion Rules . . . 8
6.3 Opcode Category Mapping . . . 13
6.4 Lowering Algorithms (Pseudo-code) . . . 14
6.5 NEF Assembly Workflow . . . 15
6.6 Manifest Generation Workflow . . . 15
6.7 NEF Binary Layout . . . 16
6.8 NEF Script Hex Example . . . 16
6.9 Manifest Schema Reference . . . 16

6.10 Mapping Detail Reference . . . 16

7 Data and Metadata Structures **17**
7.1 Compiler Configuration and Context . . . 17
7.2 Diagnostic Contracts . . . 18

8 Control Flow, State Management, and Runtime Coordination **18**
8.1 Compiler State Machine . . . 18
8.2 Runtime Layering . . . 19
8.3 Execution Overrides . . . 19

9 Optimization and Analysis Strategy **20**

10 Artifact Production and Interfaces **20**
10.1 Artifact Catalog . . . 20
10.2 Manifest Construction Rules . . . 20

11 Storage and Mapping Design **21**
11.1 Mapping and Indexed Storage Lowering . . . 21
11.2 Memory Layout Assumptions . . . 21
11.3 Storage Slot Hashing Diagram . . . 21

12 Diagnostics, Security, and Quality Attributes **22**
12.1 Diagnostic Strategy . . . 22
12.2 Security Considerations . . . 22
12.3 Security Posture and Monitoring . . . 22
12.4 Telemetry and Metrics Export . . . 22

13 Validation and Compliance **23**
13.1 Test Strategy . . . 23
13.2 Test Matrix and Coverage Goals . . . 24
13.3 Continuous Integration Flow . . . 24
13.4 CI Artifact Summary . . . 25
13.5 CI Command Summary . . . 25
13.6 Sample CI Pipeline . . . 25
13.7 Compile Summary Artifact . . . 26
13.8 Traceability Matrix . . . 26

14 Assumptions and Future Work **26**
14.1 Planned Enhancements . . . 27
14.2 Cross-Chain Compatibility Considerations . . . 27
14.3 Roadmap Milestones . . . 27

15 Conclusion **28**

1 Scope and Purpose

This specification defines the design of the Neo Solidity Compiler (*NSC*), a toolchain that ingests Solidity 0.8.x sources, leverages a Yul-centric pipeline, and emits Neo N3 artifacts (`.nef`,

`.manifest.json`, ABI metadata, and optional debug bundles). The focus is on architectural correctness, subsystem contracts, and quality attributes needed to reach the "complete, production-ready" bar stated in the project README. Implementation details (individual algorithms, data-structure internals) are intentionally out-of-scope.

1.1 Objectives

- Preserve EVM-visible semantics so Solidity code behaves identically on NeoVM when ABI inputs, storage, and events are equivalent.
- Provide a deterministic, testable compilation pipeline with explicit stage boundaries to support certification, auditing, and tooling integration.
- Capture Neo-specific design requirements: stack discipline, syscall encoding, manifest population, and devpack alignment.
- Document quality and validation responsibilities to ensure a consistent standard for future contributions.

2 References and Terminology

2.1 Normative References

Table 1: Normative reference set

ID	Artifact	Relevance
R1	`docs/ARCHITECTURE.md`	Defines baseline architecture, pipeline responsibilities, and sequencing for the production-grade toolchain.
R2	`compiler.go`	Hosts the authoritative description of driver phases, configuration knobs, and result surfaces.
R3	`YUL_TO_NEOVM_COMPILATION_STRATEGY.md`	Provides the strategy for parsing, normalizing, analysing, and emitting NeoVM bytecode from Yul IR.
R4	`docs/mapping_lowering_design.md`	Captures the required design for mapping and indexed storage lowering onto Neo storage primitives.
R5	Repository README	Specifies supported toolchain targets, optimization levels, Neo integration targets, and developer-experience requirements.

2.2 Terminology

Canonical IR The SSA-like, Neo-annotated intermediate form consumed by semantic analysis, optimisation, and code generation.

DevPack The Neo runtime bindings and Solidity helper surface that map Solidity calls to Neo syscalls, permissions, and native contracts.

Manifest The JSON artifact that records ABI, permissions, supported standards, and metadata required by Neo CLI and runtime.

Runtime Manager The compiler subsystem that links generated bytecode with runtime stubs, applies policy (execution overrides, metadata), and emits NEF/manifest structures.

Tooling Adapter Any CLI or IDE integration (Hardhat plugin, Foundry adapter, `neo-solc`) that feeds sources into the compiler and consumes diagnostics.

3 System Objectives and Constraints

Table 2: Primary objectives, constraints, and associated design responses

Objective / Constraint	Source	Design Response
Solidity 0.8.x compatibility with "complete" language feature coverage.	README Section "Core Capabilities"	Integrate `solang` frontend for syntax & semantic parity, normalize to canonical IR preserving source mapping, and extend runtime to cover ABI encoding and event emission.
Full Neo N3 deployment outputs (`.nef`, `.manifest.json`) plus debug metadata.	README Section "Neo N3 Native"	Artifact Builder stage emits NEF (CRC32, tokens), manifest (permissions, standards) and optional debug bundles tied to `CompilationResult.DebugInfo`.
Optimization levels `-O0` through `-O3` with gas-sensitive passes.	README Section "Performance & Optimization"	Optimization Engine applies ordered pass suites (constant folding, DCE, stack reduction, Neo-specific peepholes) gated by `CompilerConfig.OptimizationLevel`.
Deterministic diagnostics and metadata for auditing (400+ tests, VM validation).	README "Security & Quality" and R1 Section "Testing & Validation"	Stage-specific error collection (`ErrorCollector`), structured warnings, and CI matrix requiring unit, integration, and VM regression suites.
Rust 1.70+, Neo N3 3.0+, CLI/Node tooling availability.	README "Installation"	Build scripts assume Rust toolchain, dotnet runtime for Neo runtime library, and Node for tooling packages.

4 Architectural Overview

The compiler is organized as a sequence of pure, inspectable stages driven by the Go-based orchestration in `compiler.go`. Each stage consumes and produces well-defined data structures stored within `CompilerContext`. Figure 1 visualizes the nominal flow.

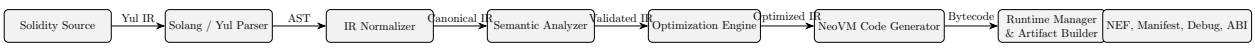

Figure 1: Compiler pipeline overview

4.1 Stage Contracts

Table 3: Pipeline stage responsibilities and invariants

Stage	Responsibilities	Invariants / Outputs
Frontend	Invoke `solang` or Yul parser, ingest diagnostics, build `YulAST`.	AST nodes annotated with `SourcePosition`, metadata includes compiler version and source identifiers.
IR Normalizer	Convert Solang/Yul AST to canonical IR with SSA-like temporaries, explicit control flow.	All identifiers bound, control flow reduced to blocks and jumps, metadata retains traceability (R3 §4).
Semantic Analysis	Build symbol/type tables, enforce Solidity and Neo constraints (storage layout, supported widths, mapping support).	`TypeTable` populated, warnings/errors classified, runtime safety diagnostics recorded.
Optimization	Apply ordered pass list conditioned on `OptimizationLevel`.	No pass mutates IR outside its ownership, change tracking feeds statistics, stack depth metrics stored for codegen.
Code Generation	Emit NeoVM opcodes, manage evaluation and alternative stacks, encode syscalls and control transfer.	Instructions annotated with stack-before/after counts (`InstructionInfo`), label fixups resolved.
Runtime Manager	Attach runtime stubs, compute gas estimates, serialize NEF/manifest, optionally create debug info.	Artifact builder ensures NEF CRC32, manifest compliance, debug maps align with `SourceMap`.

5 Component Specifications

5.1 Frontend Integration

- **Inputs:** Solidity sources, compiler flags, metadata describing target NeoVM version.
- **Processing:** Calls Solang to parse Solidity into Yul IR, then feeds Yul text into the internal lexer/parser (R2) to construct `YulAST`. Lexing includes classification of built-ins (arithmetic, memory, storage, control) to support later optimisation heuristics.
- **Outputs:** AST annotated with source file/line references; initial diagnostics from Solang forwarded through `ErrorCollector`.
- **Dependencies:** Solang crate, `YulLexer`, `YulParser`.
- **Quality Gates:** Empty sources forbidden, keyword table alignment validated on initialisation, parser errors halt compilation before normalization.

5.2 Canonical IR and Normalizer

- **Purpose:** Provide a deterministic IR that normalizes control flow, imposes SSA-style temporaries, expands built-ins, and records explicit stack/memory effects.
- **Key Behaviors:** Pass suite described in R3 §4 converts for-loops into jump-based loops, rewrites switch statements into cascaded comparisons, and ensures each variable assignment

produces a unique identifier.

- **Metadata:** Maintains source-to-node mapping for downstream debug info.
- **Interface:** `Normalizer.Normalize(ast)` returns IR graph or emits normalization errors categorized by phase.

5.3 Semantic Analysis

- **Scope:** Symbol table construction, type checking, storage layout verification, runtime safety diagnostics (re-entrancy, permission hints per R1).
- **Responsibilities:** Validate mappings (per R4), enforce Neo-supported numeric widths and storage annotations, build metadata for ABI generation (events, functions, state variables).
- **Outputs:** `analysisResult` containing warnings, errors, and typed IR. Feeds into `CompilationResult.Warnings`.
- **Failure Modes:** Missing symbols, unsupported types, invalid storage qualifiers; all must be reported with line/column metadata.

5.4 Optimization Engine

- **Configuration:** Driven by `CompilerConfig.OptimizationLevel`, enabling passes cumulatively from `-O0` (none) to `-O3` (all).
- **Pass Library:** Constant folding/propagation; dead code elimination; bounded inlining; stack height reduction; NeoVM-specific peepholes (merge PUSH sequences, syscall hoisting); optional bounds-check insertion when `EnableBoundsChecking` is true.
- **Accounting:** Each pass records change statistics into the CompilationStats.OptimizationsPassed counter; the pass manager records before/after IR snapshots for debugging.
- **Constraints:** Passes must preserve semantic equivalence, keep stack effect annotations valid, and honor max stack depth budget.

5.5 NeoVM Code Generator

- **Responsibilities:** Lower canonical IR instructions to NeoVM opcodes (`emit_ir_function`, `emit_serialize_key`, etc.), allocate locals/arguments via `INITSLOT`, and encode syscalls with proper interop IDs (storage, crypto, runtime).
- **Stack Discipline:** Maintains explicit stack-before/after counts stored in `InstructionInfo`; enforces max depth (`CompilerConfig.MaxStackDepth`).
- **Control Flow:** All jumps resolved via label fixups; structured exception regions inserted when runtime metadata demands it.
- **Storage Access:** Mapping lowering follows R4 requirements (key serialization, SHA256 hashing, context management) and uses helper routines to ensure consistent key derivation.

5.6 Runtime Manager and Artifact Builder

- **Runtime Integration:** Links generated bytecode with runtime support (ABI decoder, dispatcher), injects metadata (method table, events), and applies execution overrides via `NeoRuntime::execute_with_overrides`.

- **Artifact Generation:** Produces NEF with method tokens and CRC32 checksum, manifest with ABI/permissions/supported standards, ABI JSON for developer tooling, and optional debug bundles (`DebugInformation`).
- **Policy Hooks:** Applies gas estimates, ensures features flags (storage, dynamic invoke) align with compiled code, and ensures `CallFlags` usage matches manifest permissions.

5.7 Tooling Surfaces

- **CLI (`neo-solc`):** Accepts Sol sources, emits outputs, exposes flags for optimizations, debug info, artifact selection.
- **Hardhat / Foundry Adapters:** Wrap CLI invocation, collect `compile_summary.csv`, and map Neo diagnostics to IDE annotations.
- **Debugging Support:** Source maps, function/variable maps for breakpoints (per `DebugInformation`).

5.8 Hardhat Integration Example

```
// hardhat.config.js
module.exports = {
  solidity: "0.8.19",
  networks: {
    neo: {
      url: "http://localhost:50012",
      compiler: {
        path: "target/release/neo-solc",
        options: ["-O2", "--debug"]
      }
    }
  },
  neoSolidity: {
    outputDir: "build/neo",
    formats: ["nef", "manifest", "abi"]
  }
};
```

Hardhat tasks invoke `neo-solc` with project-specific options, then copy NEF/manifest outputs into deployment scripts.

5.9 Foundry Integration Notes

- Configure `foundry.toml` to point compiler commands at `neo-solc` for specific profiles. Custom build scripts invoke the CLI for each Solidity contract and capture NEF/manifest outputs in `out/neo`.
- Foundry tests can execute NEF artifacts via a Neo VM runner (e.g., hooking `scripts/run`$_v m_t ests.sh)tocompareEV
$readableJSONsotheIDEsurfacesline/columnerrorsinSolidityfiles.$

5.10 DevPack Interface Summary

Table 4: Key devpack entry points

Module	Interface	Usage in Compiler
`System.Storage`	`GetContext`, `Get`, `Put`	Storage lowering uses these syscalls for state access, ensuring context is fetched once per function when possible.
`System.Runtime`	`Notify`, `Log`, `GetCallingScriptHash`	Event emission, logging, and caller resolution (`msg.sender` semantics).
`System.Contract`	`Call`, `CallEx`	Backing for external Solidity calls when mapping to native contracts.
`System.Crypto`	`SHA256`, `RIPEMD160`	Mapping key hashing, signature checks, cryptographic helpers.
`System.Blockchain`	`GetHeight`, `GetHeader`	Implements Solidity environmental opcodes (block number, timestamp) when ExecutionOverrides are not provided.

5.11 DevPack API Details

Table 5: Storage and Runtime helpers

Module	Function	Notes
`System.Storage`	`GetContext/PutContext`	Context retrieval cached per function when possible; required for all storage reads/writes.
`System.Storage`	`Get`	Returns byte arrays; compiler inserts deserialization stubs based on IR type.
`System.Storage`	`Put`	Accepts byte arrays; compiler serializes integers/structs before storing.
`System.Runtime`	`Notify`	Event emission; compiler builds topic array + ABI-encoded payload.
`System.Runtime`	`Trace/Log`	Optional instrumentation hooks described in logging section.
`System.Runtime`	`GetCallingScriptHash`	Used to implement `msg.sender`.
`System.Crypto`	`SHA256`	Mapping slot hashing and other cryptographic needs.
`System.Blockchain`	`GetHeight`, `GetHeader`	Provide block metadata when ExecutionOverrides absent.

Table 6: Native contract wrappers

Wrapper	Functionality	Manifest Implications
PolicyContract	Access to policy contracts (gas price, fee settings)	Requires manifest permission for policy hash; compiler must detect usage and add permission entries.
OracleContract	Query oracle results	Permissions include oracle hash; manifest `features.payable` often true if native oracle requires GAS.
ManagementContract	Contract management operations	Restricted to administrative contexts; CLI warns when usage detected without appropriate permissions.
LedgerContract	Block/transaction info	Typically read-only; safe flag remains true.

6 End-to-End Compilation Detail

6.1 Solidity Source to Canonical IR

1. **Solidity Intake:** `solang` parses Solidity 0.8.x units, applies Solidity semantic rules, and emits Yul-compatible IR plus inline diagnostics. Source maps, type metadata, and modifier contexts are persisted for later mapping into Neo debug info.
2. **Yul Parsing:** The internal lexer/parser (`YulLexer`, `YulParser`) tokenizes Yul IR, classifies built-ins by category, and constructs `YulAST`. Each AST node records `SourcePosition` and result-type hints so subsequent passes can reason about evaluation stack effects.
3. **Canonicalization:** The normalizer applies the rules captured in R3: convert loops into explicit jump blocks; force SSA-style temporaries; expand complex built-ins (e.g., `keccak256`) into guardable helper nodes; express control flow exclusively through labeled blocks with deterministic predecessors. During this phase, inline assembly fragments are rejected unless expressible via supported Yul primitives, ensuring deterministic lowering.
4. **Semantic Decoration:** The semantic analyzer enriches IR with type ids, storage locations, and binding information. State variables gain explicit storage slots (hashed identifiers, bit widths), while function signatures gain ABI ordinals derived from Solidity selectors. Diagnostics emitted here block compilation if storage layout, inheritance, or modifier resolution fails.

6.2 Yul-to-NeoVM Conversion Rules

General Conventions

- Evaluation stack order follows NeoVM semantics (right-most operand on top). Prior to emitting an opcode, the code generator rewrites Yul argument order where necessary so that the stack shape matches the Neo instruction contract. Temporary stack height adjustments rely on `SWAP`, `ROT`, and `DROP` sequences with bounds tracked against `MaxStackDepth`.
- Each NeoVM method starts by reserving locals with `INITSLOT` using counts derived from IR liveness analysis. Function return values are pushed onto the stack immediately before emitting `RET`; multi-return sequences follow the ABI order established in the IR metadata.
- Branch constructs materialize as labeled blocks. The generator emits placeholders for jump

offsets and resolves them after the full method body has been encoded, ensuring canonical byte offsets for manifest/debug metadata.

- Syscalls are encoded with their interop ids, pulled from the devpack metadata. The generator centralizes syscall emission so that audit trails can map IR instructions to their concrete `SYSCALL` operands.

Table 7: Representative Yul → NeoVM opcode mapping

Yul Construct	Stack Convention	NeoVM Emission Pattern
`add(a,b)`	Push b then a; result replaces both	`ADD`. Overflow behavior inherits NeoVM big-int arithmetic; if Solidity semantics require mod-2^{256}, truncation logic inserts `MOD` with constant modulus.
`mul(a,b)`, `sub(a,b)`	Same operand ordering as `add`.	Direct `MUL`, `SUB`. Signed variants (`sdiv`, `smod`) wrap operands with sign-normalization helpers before emitting `DIV`/`MOD`.
Comparison (`lt`, `gt`, `eq`)	Evaluates operands, leaves boolean (0/1).	Emits corresponding Neo instructions; signed comparisons insert `SIGN` normalization.
Bitwise (`and`, `or`, `xor`, shifts)	Operands aligned to stack order; shift counts normalized to big-endian.	Uses NeoVM bitwise opcodes; `shl`/`shr` translate to `SHL`/`SHR`; arithmetic right-shift uses helper that inspects sign bit.
`mload`/`mstore`	Memory abstraction backed by runtime-managed byte arrays.	Resolves to calls into the runtime memory manager: compute base pointer, call `System.Runtime.Serialize` or helper stubs, and use `PICKITEM`/`SETITEM` to manipulate backing arrays.
`sload`/`sstore`	Requires slot hash on stack, value for store.	Delegated to storage helpers: push storage context (`System.Storage.GetContext`), prepare slot byte array, call `System.Storage.Get`/`Put`. Mapping keys follow Section 6.10.
Control flow (`if`, `switch`, `for`)	Conditions evaluated to boolean on stack.	Emit `JMPIF`/`JMPIFNOT` plus labels. `switch` cascades comparisons with duplicated selector values and uses fall-through default blocks (mirroring the pseudo-code in R3 §5.3).
Function call `f(a,b)`	Arguments pushed reverse order, return slot reserved.	For internal functions, emit `CALL` to method offset with proper `INITSLOT`; for external ABI calls, delegate to runtime dispatcher (ABI decoder pushes arguments, selects target).
`revert`/`return`	Offsets and lengths follow ABI spec.	Use runtime helpers to format return buffers, then emit `ABORT` or `RET` with data pointer per ABI encoding rules.
`logi`	Topics/data packed on stack.	Call `System.Runtime.Notify` via devpack wrappers, ensuring topic ordering mirrors Solidity event ABI.

Specific Conversion Rules

1. **ABI Dispatcher:** Constructor and runtime objects from Yul translate into separate NeoVM

entry points. The dispatcher inspects the first 4 bytes of the calldata-equivalent buffer (Neo invocation arguments) and routes to the method offset table. Unknown selectors trigger the standard Solidity revert pattern.

2. **Memory Model:** Solidity memory abstractions map onto NeoVM arrays managed by the runtime. `calldatacopy`, `returndatacopy`, and ABI encode/decode routines call into runtime helpers that perform bounds checking and convert between little-endian Solidity expectations and Neo big-endian integers.
3. **Error Handling:** Solidity `require`/`assert` nodes become structured exception blocks. The generator emits predicates, `JMPIFNOT` gates, and a revert stub that encodes error selectors in ABI-compatible format.
4. **Stack Safety:** Each IR instruction carries an expected stack delta; the generator uses this metadata to insert spill-to-alt-stack instructions when stack depth approaches `MaxStackDepth`, ensuring NeoVM never faults due to stack overflow.
5. **Gas/Cost Hints:** While NeoVM gas differs from EVM gas, the generator annotates blocks with syscall counts and arithmetic complexity so the Runtime Manager can compute gross gas estimates stored inside `CompilationStats`.

6.3 Opcode Category Mapping

Table 8: Category-level mapping reference

Yul Category	Representative Operations	NeoVM Implementation Notes
Arithmetic	`add`, `sub`, `mul`, `exp`, `addmod`, `mulmod`	Direct opcode emission with optional normalization. Modular variants expand to exponentiation plus `MOD`. Exponentiation uses `POW` helper built atop repeated squaring macros owing to NeoVM lacking native exponent opcode.
Comparison & Boolean	`lt`, `gt`, `slt`, `eq`, `iszero`	Signed variants translate into sequences that inspect sign bit via `SIGN` and adjust operands before `LT`/`GT`. Boolean negation is `NOT` applied to 0/1 representation.
Bitwise	`and`, `or`, `xor`, `byte`, `shl`, `shr`, `sar`	Byte extraction implemented using `PICKITEM` on byte arrays returned by runtime serialization. Arithmetic shift right inserts sign-extension guard around `SHR`.
Memory	`mload`, `mstore`, `mstore8`, `msize`, `calldatacopy`	Provided via runtime-managed byte arrays. `calldatacopy` becomes a runtime call that slices the invocation argument buffer using `SUBSTR` and writes into contract-managed memory arrays.
Storage	`sload`, `sstore`, mapping/address ops	Emit `System.Storage.GetContext` followed by key hashing and `System.Storage.Get/Put`. Nested mappings rely on the slot-derivation helpers described in Section 6.10.
Environment	`gas`, `timestamp`, `caller`, `origin`, `blockhash`	Bridge to runtime-latched metadata or Neo syscalls (`System.Blockchain.GetTimestamp`, etc.). When the target attribute is not directly available (e.g., `gasprice`), shim functions return ABI-compatible placeholders documented in runtime policy.
Control Flow	`if`, `switch`, `for`, `return`, `revert`	Lower to canonical jump graphs. `return` uses runtime-managed ABI encoder; `revert` emits error-selector packaging and jumps to a shared revert stub that aborts execution with encoded data.
External Calls	`call`, `delegatecall`, `staticcall`	Neo lacks EVM-style calls, so runtime stubs convert these operations into native-contract invocations or cross-contract dynamic invoke sequences, setting `CallFlags` according to op semantics (e.g., `delegatecall` forbids state writes).
Events	`log0`–`log4`	Map to `System.Runtime.Notify`, constructing an array of topics and the ABI-encoded data payload. Indexed parameters become topics, matching Solidity's event encoding rules.

6.4 Lowering Algorithms (Pseudo-code)

Mapping Load

```
procedure emit_load_mapping(state_index, key_types):
    # stack: [... outer_key, inner_key]
    for key_type in key_types (outermost -> innermost):
        emit_serialize_key(key_type)   # consumes key, pushes canonical bytes
        push slot_hash[state_index]    # 32-byte base slot (first iteration)
        SWAP; CAT                      # concatenate key || slot
        SYSCALL System.Crypto.SHA256   # new slot on stack
    SYSCALL System.Storage.GetContext
    SWAP                              # ensure stack = [context, slot]
    SYSCALL System.Storage.Get        # leaves value
```

Conditional Lowering

```
procedure lower_if(condition_block, true_block):
    emit evaluate(condition_block)        # leaves boolean on stack
    create label skip_label
    JMPIFNOT skip_label
    emit block(true_block)
    place label skip_label
```

Switch Lowering

```
procedure lower_switch(expr, cases, default_block):
    emit evaluate(expr)
    DUP
    for case in cases:
        DUP
        PUSH case.value
        EQUAL
        JMPIF case.label
    DROP
    JMP default_label
label blocks emit bodies accordingly
```

ABI Dispatcher Skeleton

```
procedure dispatcher():
    selector := decode_first_4_bytes(invocation_args)
    switch selector:
        case HASH("transfer(address,uint256)"):
            route_to transfer_impl
        case HASH("balanceOf(address)"):
            route_to balance_impl
        default:
            revert_with_error("Unknown selector")
```

6.5 NEF Assembly Workflow

1. **Method Table Construction:** Each callable entry (public functions, event handlers, static constructors) becomes a method token entry. The compiler records name, parameter count, return count, and instruction pointer offset.
2. **Script Buffer Finalization:** After codegen resolves jumps, the byte vector is frozen. Debug metadata (source map, instruction map) stores final offsets so the NEF can point back to Solidity lines.
3. **Checksum and Header:** The NEF header embeds compiler information, script length, and checksum. The compiler computes CRC32 over the serialized script and inserts it into the header, matching Neo's validation requirements.
4. **Token Serialization:** Method tokens, ABI descriptions, and additional data (e.g., storage ref counters) serialize into the NEF structure per Neo's specification. Optional `nef.debug.json` is generated in parallel when debug info is enabled.
5. **Validation Pass:** Before returning artifacts, the Runtime Manager re-parses the NEF buffer to ensure it meets NeoVM constraints (no unresolved jumps, method offsets within range, CRC match). Failures at this stage are surfaced as Runtime Integration errors.

6.6 Manifest Generation Workflow

1. **ABI Extraction:** Symbol tables provide function names, parameter types, visibility, and event topics. Functions become manifest `methods` entries with offsets and safe parameter metadata; events become `events` entries with indexed flag information mirroring Solidity's `indexed` attributes.
2. **Permission Derivation:** The code generator records every syscall/native contract reference. The builder groups these references by contract hash or interop service and emits manifest `permissions`. Devpack annotations determine default groups for runtime services (storage, notifications, crypto).
3. **Supported Standards Detection:** Interface conformance (e.g., NEP-11/17/24) is inferred by checking whether contract symbols implement the entire required method set plus event signatures. When matched, the manifest's `supportedstandards` list receives the corresponding NEP identifier.
4. **Features Flags:** The analyzer marks whether storage, dynamic invoke, and payable semantics are used. These booleans map directly to manifest `features.storage`, `features.dynamicInvoke`, and `features.payable`. Payable is enabled when ABI parsing finds functions marked `payable` or implicit constructors receiving assets.
5. **Extra Metadata:** Compilation metadata (compiler version, optimization level, source hash) is attached under the manifest's `extra` or `description` fields so downstream tooling can audit builds.

6.7 NEF Binary Layout

Table 9: NEF structural fields

Field	Size (bytes)	Description
Magic	4	Constant header identifying NEF files.
Compiler	32	ASCII identifier (e.g., "neo-solidity v1.0.0").
Version	2	NEF version supported by target NeoVM.
CheckSum	4	CRC32 calculated over the entire script payload.
Script Length	2	Size of the bytecode blob that follows.
Script	variable	NeoVM bytecode emitted by code generator.
Tokens	variable	Method tokens: name, offset, parameter count.
Metadata	variable	Optional debug info pointer, reserve data.

6.8 NEF Script Hex Example

```
00000001 6e656f2d736f6c6964697479207631e302e30  # compiler string
0003                                             # NEF version
4f2a6bc1                                         # CRC32
012c                                             # script length (300 bytes)
52 6c 76 68 ...                                  # NeoVM bytecode (truncated)
0200                                             # method token count
077472616e73666572 3402 0100                     # token entry: name, offset, params/returns
...                                              # additional metadata/debug references
```

6.9 Manifest Schema Reference

Table 10: Manifest key fields

Field	Type	Description
`name`	string	Solidity contract name; taken from compilation metadata.
`groups`	array	Future extension (security groups); empty unless specified.
`supportedstandards`	array	List of NEP identifiers automatically inferred.
`abi.methods`	array	Methods with name, parameters, return types, offset.
`abi.events`	array	Events with name, parameters, indexed flags.
`permissions`	array	Each entry references contract hash or wildcard plus allowed methods.
`trusts`	array	Contracts implicitly trusted (rare; enforced via config).
`features.storage`	bool	True when state writes present.
`features.dynamicInvoke`	bool	True if runtime emits dynamic invoke syscalls.
`features.payable`	bool	True if methods accept assets.
`extra`	object/string	Embeds `CompilationMetadata` snapshot.

6.10 Mapping Detail Reference

Mapping lowering follows R4 and Section 6.10 ensures runtime determinism:

- Slot hashes are pre-computed at semantic-analysis time using the SHA256 of the state variable name. Nested mappings iterate hashing per key.
- Key serialization leverages runtime helpers that convert integers to fixed-width big-endian byte arrays, addresses to 20-byte sequences, and strings to UTF-8 without length prefixes. These helpers return canonical byte arrays consumed by the hashing loop.
- Generated code orders keys so the outermost key is on the top of the stack just prior to slot derivation. During hashing, the code alternates between concatenation (`CAT`) and `System.Crypto.SHA256`.
- Load/store helpers manage stack layout: `emit_load_mapping` expects keys on stack, pushes storage context, and calls `System.Storage.Get` while preserving evaluation order. Stores ensure values remain on stack until the final `Put` invocation consumes them.

7 Data and Metadata Structures

7.1 Compiler Configuration and Context

Table 11: Configuration surface (`CompilerConfig`)

Field	Type	Description
`OptimizationLevel`	`int`	Pass suite level (0 to 3); determines aggressive transformations permitted.
`TargetNeoVMVersion`	`string`	Declares target VM (3.0–3.5+); influences syscall set and manifest compatibility.
`EnableBoundsChecking`	`bool`	Inserts runtime bounds checks in codegen for memory/array accesses when true.
`EnableDebugInfo`	`bool`	Enables generation of `DebugInformation`.
`MaxStackDepth`	`int`	Upper bound for evaluation stack depth; codegen must enforce.
`MemoryLimit`	`int64`	Max memory used during compilation (defensive against large contracts).
`CompilerFlags`	`[]string`	Additional feature toggles, e.g., experimental passes or manifest policies.

Table 12: Core context and result records

Structure	Key Fields	Purpose
`CompilerContext`	`SourceMap`, `SymbolTable`, `TypeTable`, `LabelCounter`, `ErrorCollector`, `CompilationMetadata`	Shared state across stages; ensures consistent symbol/type resolution and diagnostic aggregation.
`CompilationResult`	`Contract`, `Warnings`, `Errors`, `Statistics`, `DebugInfo`	Final deliverable to tooling; either contains emitted artifacts or records failure causes per phase.
`CompilationMetadata`	`CompilerVersion`, `CompilationTime`, `SourceHash`, `Statistics`	Audit trail for builds; used in debug bundles and manifest extra metadata.
`DebugInformation`	`SourceMap`, `FunctionMap`, `VariableMap`, `InstructionMap`	Enables IDEs/debuggers to map bytecode offsets back to source constructs.

7.2 Diagnostic Contracts

- **Severity Levels:** Errors (fatal) and warnings (non-fatal) per `CompilerError` and `CompilerWarning`.
- **Phase Tagging:** Every diagnostic ties back to the emitting stage (*Parsing*, *Normalization*, *Static Analysis*, etc.) to support stage-by-stage gating and analytics.
- **Traceability:** All diagnostics must carry source locations when available; emitted artifacts maintain `SourceMap` keyed by bytecode offsets.

8 Control Flow, State Management, and Runtime Coordination

8.1 Compiler State Machine

Table 13: State transitions

State	Entry Condition	Exit Condition
Initialized	`NewYulToNeoCompiler` invoked; context allocated.	Parser consumes first token from source.
Parsed	`Parser.Parse` succeeded.	`Normalizer.Normalize` completes; invalid AST halts pipeline.
Normalized	Canonical IR built.	Semantic analyzer validates and populates symbol/type tables.
Analyzed	Semantic checks completed; warnings recorded.	Optimization passes applied successfully.

State	Entry Condition	Exit Condition
Optimized	All enabled passes report success; IR annotated with stack metrics.	Code generator emits bytecode without exceeding stack/memory constraints.
Generated	Bytecode exists; instruction metadata built.	Runtime manager attaches runtime artifacts, NEF, manifest.
Finalized	Contract artifacts available; debug info optional.	Compilation result returned to caller or CLI for persistence.

8.2 Runtime Layering

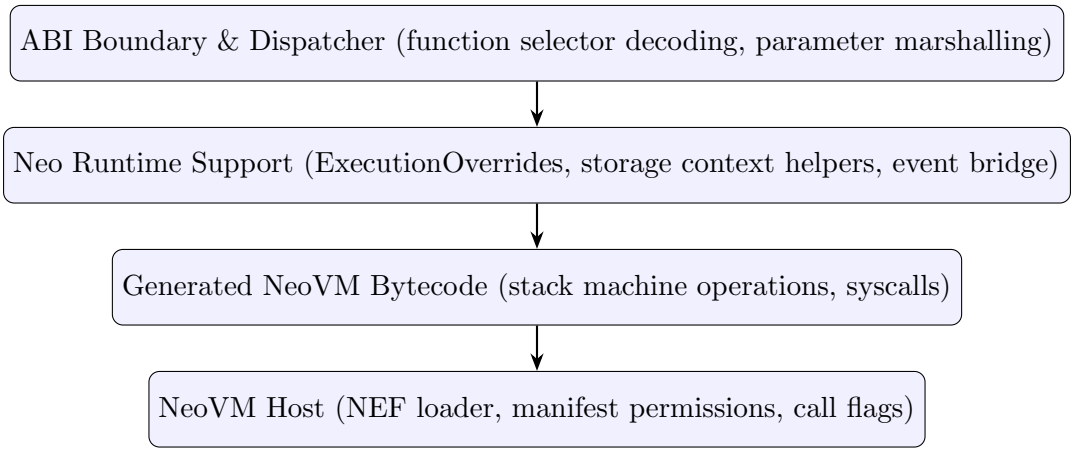

Figure 2: Runtime layering and responsibilities

8.3 Execution Overrides

- **Design Intent:** Allow deterministic injection of block height, timestamp, and calling script hash for tests without mutating global runtime state (per README "Runtime Metadata Overrides").
- **Mechanism:** `NeoRuntime::execute_with_overrides` accepts `ExecutionOverrides`, executes one invocation, and returns `ExecutionResult` carrying `ExecutionMetadata`.
- **Constraints:** Overrides apply to a single execution, clear afterwards, and default to `RuntimeConfig` otherwise.

9 Optimization and Analysis Strategy

Table 14: Optimization pass catalog

Pass	Trigger Level	Notes
Constant Folding / Propagation	`-O1`–`-O3`	Simplifies arithmetic, comparisons, and branches when operands literalized by frontend.
Dead Code Elimination	`-O1`–`-O3`	Removes unreachable blocks and unused assignments; ensures elimination respects event side-effects.
Function Inlining	`-O2`–`-O3`	Applies heuristics based on stack depth budget and call frequency; avoids recursion.
Stack Height Reduction	`-O2`–`-O3`	Peephole rewriting to minimize DUP/SWAP sequences before codegen (R3 §6.4).
Neo-Specific Peepholes	`-O3`	Merges adjacent `PUSH` ops, hoists repeated syscall operands, balances alt-stack usage.
Bounds Check Injection	Optional flag	Ensures safe memory and array access; complements `EnableBoundsChecking`.
Gas Estimation	Always-on	Tracks syscall/gas usage for reporting; influences manifest `features`.

10 Artifact Production and Interfaces

10.1 Artifact Catalog

Table 15: Generated artifacts and consumers

Artifact	Contents	Consumers / Notes
NEF (`*.nef`)	Bytecode, method tokens, checksum.	Neo CLI, NeoExpress, runtime host; must align with target Neo version.
Manifest (`*.manifest.json`)	ABI signatures, events, permissions, supported standards, `features`.	Deployment pipelines, policy validators; derived from semantic metadata.
ABI JSON	Function/event descriptors for tooling.	Hardhat plugin, Foundry adapter, language servers.
Debug Bundle (`nef.debug.json`)	Source map, instruction map, variable map.	Debuggers and IDE breakpoints (per README "Rich Debugging").
Compile Summary (`compile_summary.csv`)	Batch compilation outcomes.	CI reporting, regression dashboards (README "Batch Compilation").

10.2 Manifest Construction Rules

- **ABI:** Derived from symbol tables (functions, events, state variables); honours overload resolution and parameter types.

- **Permissions:** For each syscall/native contract usage discovered during codegen, manifest must include corresponding `permissions` entries.
- **Supported Standards:** NEP-11/17/24 flags automatically populated when devpack contracts or interfaces detected in sources.
- **Features:** `storage`, `dynamicInvoke`, `payable` toggles computed based on IR analysis and runtime instrumentation.

11 Storage and Mapping Design

11.1 Mapping and Indexed Storage Lowering

The compiler must follow the plan in R4 when lowering mappings and struct-field references:

1. **Representation:** Extend IR types with ValueType::Mapping and instructions (`LoadMapping`, `StoreMapping`, `AddressOfMapping`, `LoadStructField`, `StoreStructField`).
2. **Key Serialization:** Serialize keys using canonical rules (big-endian padded integers, UTF-8 strings, raw bytes for addresses). Keys pushed in reverse order such that the outermost key sits on top of the stack before hashing.
3. **Slot Derivation:** For each key, compute SHA256($key\_bytes \| slot$) starting from the precomputed state-variable slot hash.
4. **Syscall Ordering:** After slot derivation, push storage context via System.Storage.GetContext, reorder stack to match syscall signatures, and invoke System.Storage.Get or `Put`.
5. **Diagnostics:** Reject unsupported value types, warn when values contain dynamic arrays (future work), and trace stack shapes for debugging.

11.2 Memory Layout Assumptions

- Persistent state resides in Neo storage, addressed via 32-byte slot hashes.
- Evaluation stack hosts temporaries, call arguments, and return slots; alt-stack used sparingly for spills.
- Runtime-managed buffers (ABI encoding/decoding) obey Neo serialization routines to remain deterministic across nodes.

11.3 Storage Slot Hashing Diagram

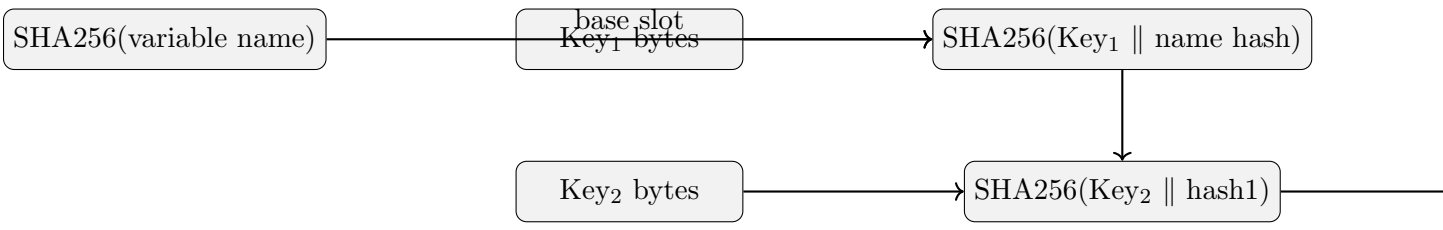

Figure 3: Nested mapping slot derivation

12 Diagnostics, Security, and Quality Attributes

12.1 Diagnostic Strategy

- **Phase-specific messaging:** Errors/warnings attach phase tags to support gating (e.g., CLI can stop after parsing warnings if `--fail-on-warning` is set).
- **Source Mapping:** `buildSourceMap`, `buildFunctionMap`, `buildVariableMap`, and `buildInstructionMap` ensure every emitted instruction can be traced back to a source region.
- **Logging:** Each driver stage logs start/end markers (see R2 logging statements) for observability.

12.2 Security Considerations

- **Compiler Security:** Input validation, resource limits (`MemoryLimit`), and deterministic output guard against compiler-level DoS or inconsistent builds (R3 §13.1).
- **Runtime Security:** Bounds checking, stack depth monitoring, and call validation (call flags vs manifest) uphold Neo runtime safety expectations (R3 §13.2).
- **Metadata Integrity:** CRC32 for NEF, manifest hashing, and recorded `SourceHash` ensure artifact provenance.

12.3 Security Posture and Monitoring

- **Audit Trail:** Every compilation emits `CompilationMetadata` (compiler version, source hash, configuration) stored in manifests and debug bundles, enabling auditors to verify the exact toolchain used for deployment.
- **Logging Integration:** Structured logs produced by runtime instrumentation can feed into SIEM systems to monitor contract behavior, detect abnormal storage writes, and correlate revert reasons across deployments.
- **Supply Chain Controls:** Release artifacts (binaries, NEFs, spec PDF) include checksums and signatures. CI enforces reproducible builds by pinning toolchain versions (Rust, Go, dotnet).
- **Monitoring Hooks:** The compiler can emit optional health-check endpoints or telemetry events summarizing compilation errors/warnings, enabling teams to monitor code health across repositories.
- **Audit Readiness:** The spec, manifest metadata, and traceability matrix collectively provide auditors with a single reference covering architecture, testing, and configuration surfaces. Teams are encouraged to keep this spec updated per release.

12.4 Telemetry and Metrics Export

- **Compilation Metrics:** The CLI can emit metrics (JSON or Prometheus exposition) summarizing `CompilationStats` (compile time, bytecode size, optimization counts) so build systems can monitor regressions over time.
- **Runtime Metrics:** Instrumentation hooks allow exporting syscall counts, storage operations, and stack depth usage during VM regression tests. These metrics help tune optimization passes and detect abnormal patterns.

- **Environment Compatibility:** The same telemetry endpoints can differentiate between local, TestNet, and MainNet builds by tagging metrics with environment labels derived from `ExecutionOverrides` or CLI flags.
- **Collection Pipeline:** Metrics are typically written to `build/reports/metrics.json` and optionally pushed to monitoring systems. CI jobs archive these reports alongside NEF/manifest artifacts for future analysis.

13 Validation and Compliance

13.1 Test Strategy

Table 16: Validation layers

Layer	Coverage	Evidence / Tooling
Unit Tests	Lexer/parser, semantic rules, optimizer passes, mapping lowering helpers.	Rust/Go test suites under `src`, `tests`, targeting canonical IR transformations.
Integration Tests	Reference contracts (NEP-11/17/24, Uniswap suite) compiled end-to-end.	Batch compilation command (README §"Batch Compilation") with `compile_summary.csv` showing 132 successes baseline.
VM Regression	Execute generated NEF in Neo VM runner or NeoExpress, validate observable behavior matches Solidity expectations.	`NeoRuntime::execute_with_overrides` harness plus NEP sample manifests.
Static/Quality Analysis	Linting (`cargo fmt`, `clippy`), code metrics, security scans.	GitHub Actions matrix as described in R1 "Testing & Validation".

13.2 Test Matrix and Coverage Goals

Table 17: Representative contract suite coverage

Suite	Contracts	Purpose
Core NEP Set	NEP-11, NEP-17, NEP-24 reference implementations	Validates token standards, event emission, mapping storage correctness.
Uniswap Reference	130+ Solidity contracts in `uniswap/`	Exercises complex inheritance, libraries, ABI interactions; captures real-world deployment scenarios.
Runtime Scenarios	Synthetic contracts covering syscalls, storage edge cases, exception handling	Ensures runtime helpers (ExecutionOverrides, storage hashing) behave correctly.
DevPack Examples	Contracts relying on Neo native contracts (oracle, policy, management)	Verifies interop IDs, manifest permissions, and devpack wrappers.

13.3 Continuous Integration Flow

- **Static Checks:** `cargo fmt`, `clippy`, Go formatters, and TypeScript linting (where applicable) run in parallel jobs.
- **Unit/Integration Tests:** Rust and Go unit tests execute ahead of integration suites. Integration stage compiles the NEP set plus Uniswap suite, persisting `compile_summary.csv` artifacts for failure triage.
- **VM Regression:** Selected NEF outputs run inside NeoExpress or the Neo VM runner to assert runtime behavior (deploy + method invocation). Failures break the pipeline and surface logs with bytecode diffs.
- **Artifact Validation:** Generated NEF/manifest pairs undergo schema validation, checksum verification, and manifest-permission sanity checks.
- **Publishing:** On tagged releases, the pipeline packages binaries, attaches NEF/manifest samples, and uploads the refreshed PDF spec as a release artifact for consumer reference.

13.4 CI Artifact Summary

Table 18: Uploaded artifacts per pipeline run

Artifact	Location	Purpose
`compile_summary.csv`	`build/reports`	Tracks contract compilation outcomes; consumed by dashboards.
`build/nefs/*.nef`	`build/nefs`	Compiled bytecode for inspection or deployment tests.
`build/manifests/*.manifest.json`	`build/manifests`	Manifest outputs paired with NEFs.
`spec/neo_solidity_compiler_design_spec.pdf`	Release artifacts	Provides the latest formal spec to consumers.
`logs/*.txt`	`build/logs`	Contains integration/VM run logs for failure triage.

13.5 CI Command Summary

Table 19: Key CI commands per stage

Stage	Command	Purpose
Format/Lint	`cargo fmt -- --check`, `cargo clippy`, `go fmt ./...`	Ensures code style consistency before tests.
Unit Tests	`cargo test --all`, `go test ./...`	Verifies lexer/parser/codegen helpers and Go runtime pieces.
Integration Tests	`scripts/compile_nep_suite.sh`, `scripts/compile_uniswap.sh`	Produces NEF/manifest outputs and `compile_summary.csv`.
VM Regression	`scripts/run_vm_tests.sh` (NeoExpress)	Executes NEFs inside a VM runner, capturing logs.
Artifact Packaging	`scripts/package_release.sh`	Bundles binaries, NEFs, manifests, and spec PDF for release.

13.6 Sample CI Pipeline

```
name: CI
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-rust@v1
        with: {rust-version: 1.70}
      - run: cargo fmt -- --check
      - run: cargo clippy -- -D warnings
      - run: cargo test --all
      - run: scripts/compile_nep_suite.sh
```

```
- run: scripts/compile_uniswap.sh
- run: scripts/run_vm_tests.sh
- name: Upload artifacts
  uses: actions/upload-artifact@v4
  with:
    name: neo-solidity-artifacts
    path: |
      build/nefs
      build/manifests
      build/reports/compile_summary.csv
      spec/neo_solidity_compiler_design_spec.pdf
```

13.7 Compile Summary Artifact

```
contract, result, warnings, errors, elapsed_ms
NEP17.sol, success, 0, 0, 352
UniswapV2Router.sol, success, 1, 0, 812
UniswapV2Pair.sol, success, 0, 0, 644
OracleExample.sol, failure, 0, 1, 115
```

CI jobs upload `compile_summary.csv` so downstream dashboards can trend success rates and highlight regressions.

13.8 Traceability Matrix

Table 20: Requirement traceability

Requirement	Satisfied By	Verification
Solidity feature parity	Frontend + Semantic Analyzer components.	Differential testing vs Solang diagnostics, integration suite results.
Neo artifact emission	Runtime Manager + Artifact Builder.	Inspection of NEF/manifest outputs, deployment via Neo CLI.
Optimization levels	Optimization Engine configuration.	Unit tests per pass, CLI flag tests, compilation statistics.
Debugging support	DebugInformation builders.	IDE integration tests ensuring breakpoints map to instructions.
Mapping lowering correctness	Code generator helpers per R4.	Unit tests with NEP-17 storage patterns, VM validation of mapping operations.
Runtime metadata overrides	`ExecutionOverrides` contract in runtime.	Tests invoking `execute_with_overrides`, verifying metadata payload.

14 Assumptions and Future Work

- Licensing resolution for Solang remains open (R1 “Open Questions”); if unresolved, fallback to consuming `solc` JSON outputs.

- Storage layout alignment with Neo key-value model is partially addressed (mappings complete; dynamic arrays scheduled as follow-up).
- Gas accounting strategy may evolve as Neo provides more precise estimators.
- Debug info format should be reconciled with Neo debugger expectations once specifications are finalized.

14.1 Planned Enhancements

- **Dynamic Array Lowering:** Extend the mapping/storage infrastructure to cover packed dynamic arrays, including slice operations and bounds-check optimizations.
- **Formal Debug Spec Alignment:** Collaborate with Neo debugger maintainers to lock down the `nef.debug.json` schema, ensuring IDE tooling can rely on stable contracts.
- **Incremental Compilation:** Investigate caching canonical IR fragments to reduce compile times for iterative development workflows.
- **DevPack API Coverage:** Document and integrate additional native contract wrappers (e.g., oracle, ledger) with exhaustive manifest permission derivation.
- **CI Enhancements:** Add mutation/fuzz testing stages and automated manifest permission linting to catch regressions earlier.

14.2 Cross-Chain Compatibility Considerations

- **EVM Parity Goals:** The compiler preserves Solidity semantics so contracts can migrate from EVM chains to Neo N3 with minimal changes. Future work includes automated manifest generation based on EVM metadata to streamline cross-chain deployments.
- **Interop Bridges:** Devpack planning includes wrappers for cross-chain bridges or relayers. Manifest permissions will need to capture native contract calls required by bridge logic.
- **ABI Compatibility:** By maintaining standard Solidity ABI encoding, tooling like Hardhat and Foundry can produce payloads compatible with Neo invocation, easing cross-chain testing and simulation.
- **Future Interop:** Longer-term, the team may add modules that read EVM bytecode directly and translate to NeoVM, enabling cross-chain verification and reducing compilation steps for existing deployments.

14.3 Roadmap Milestones

1. **Milestone 1 — Toolchain Hardening (Q1):**
   - Complete dynamic array lowering with associated tests.
   - Finalize debug spec alignment with Neo tooling team.
   - Deliver initial telemetry output and ensure CI captures metrics.
2. **Milestone 2 — DevPack Expansion (Q2):**
   - Implement additional native contract wrappers (oracle, policy, management).
   - Enhance manifest generation to automatically infer permissions from new wrappers.
   - Extend CI to cover new devpack samples and VM regression scenarios. ¡/beginitemize
   - **Milestone 3 — Cross-Chain Interop (Q3):**
     - Prototype EVM bytecode ingestion path for equivalence testing.
     - Integrate bridge-specific DevPack shims and document manifest requirements.

  - Produce interoperability guides demonstrating migration from EVM chains to Neo N3.
- **Milestone 4 — Performance & UX (Q4):**
  - Investigate incremental compilation/cache reuse to speed up iterative builds.
  - Expand CLI ergonomics (auto-discovery of project structure, better diagnostics).
  - Publish updated spec and tooling docs with each quarter's release.

15 Conclusion

This document formalizes the design of the Neo Solidity Compiler, ensuring every subsystem is articulated with clear inputs, outputs, and invariants. The specification grounds itself in the repository's architectural intent (R1), driver implementation (R2), IR strategy (R3), specialized lowering requirements (R4), and published user-facing commitments (README). Adhering to this specification guarantees that future implementation work preserves compatibility, auditability, and the high-quality developer experience demanded for Solidity-to-Neo workloads.

References

(a) R1 — `docs/ARCHITECTURE.md`: architectural overview and implementation order.
(b) R2 — `compiler.go`: Go driver, compiler context, and phase orchestration.
(c) R3 — `YUL_TO_NEOVM_COMPILATION_STRATEGY.md`: IR strategy and lowering plans.
(d) R4 — `docs/mapping_lowering_design.md`: mapping/storage lowering details.
(e) README — Repository README covering features, installation, and tooling.
(f) Additional runtime/devpack docs located under `devpack/` and `docs/`.